In [5]:

```
%pip install pillow opencv-python
```

```
Requirement already satisfied: pillow in c:\users\m\miniconda3\lib\site-packages (12.0.0)
Requirement already satisfied: opencv-python in c:\users\m\miniconda3\lib\site-packages
(5.0.0.93)
Requirement already satisfied: numpy>=2 in c:\users\m\miniconda3\lib\site-packages (from opencv-
python) (2.4.3)
Note: you may need to restart the kernel to use updated packages.
```

Step#1: Import the libraries

In [1]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

Step#2: Define Functions

In [8]:

```
def sample_image(image, factor):
    height, width = image.shape[:2]

    sampled_image = cv2.resize(
        image,
        (width // factor, height // factor),
        interpolation=cv2.INTER_NEAREST
    )

    return sampled_image


def quantize_image(image, levels):
    """
    Reduces the number of grayscale levels in the image.
    """
    quantized_image = np.floor(
        image / (256 // levels)
    ) * (256 // levels)

    quantized_image = quantized_image.astype(np.uint8)

    return quantized_image


def plot_images(original, sampled, quantized):
    """
    Plots the original, sampled, and quantized images side by side.
    """
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')

    plt.show()
```

Step#3: Import the image and set the parameters

In [5]:

```
image_path = r'C:\Users\M\Desktop\arti\lena_gray_256.tif'

sampling_factor = 14
quantization_levels = 9
original_image = cv2.imread(
    image_path,
    cv2.IMREAD_GRAYSCALE
)

if original_image is None:
    print("Error: Unable to load image")
else:
    sampled_image = sample_image(original_image, sampling_factor)
    quantized_image = quantize_image(original_image, quantization_levels)

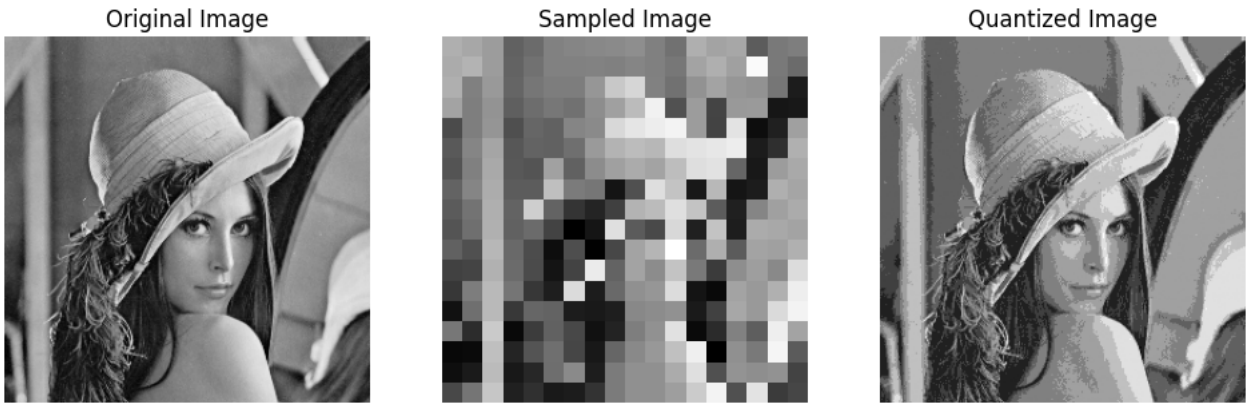
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled_image, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized_image, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')

    plt.show()
```



Arithmetic Operations

In [10]:

```
img1 = Image.open(r'C:\Users\M\Desktop\arti\lena_gray_256.tif')
img2 = Image.open(r'C:\Users\M\Desktop\arti\cameraman.tif')

resize = (400, 400)

img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1)
im2arr = np.asarray(img2)

addition = np.clip(
    im1arr.astype(np.int16) + im2arr.astype(np.int16),
    0,
    255
)

resultImage = Image.fromarray(addition.astype(np.uint8))
```

```
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img1, cmap='gray')
plt.title('Lena')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(img2, cmap='gray')
plt.title('Cameraman')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(resultImage, cmap='gray')
plt.title('Addition')
plt.axis('off')

plt.show()
```



Sets and Logical Operations

In [18]:

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img3 = Image.open(r'C:\Users\M\Desktop\arti\lena_gray_256.tif').convert('L')
img4 = Image.open(r'C:\Users\M\Desktop\arti\cameraman.tif').convert('L')

resize = (400, 400)

img3 = img3.resize(resize, Image.Resampling.LANCZOS)
img4 = img4.resize(resize, Image.Resampling.LANCZOS)

im3arr = np.asarray(img3)
im4arr = np.asarray(img4)

union = im4arr | im3arr

resultImage2 = Image.fromarray(union)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img3, cmap='gray')
plt.title('Image A')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(img4, cmap='gray')
plt.title('Image B')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(resultImage2, cmap='gray')
plt.title('Union (OR)')
```

```
plt.axis('off')

plt.show()
```



Task #1 — Sampling and Quantization

In [22]:

```
def sample_image(image, factor):
    height, width = image.shape[:2]

    sampled_image = cv2.resize(
        image,
        (width // factor, height // factor),
        interpolation=cv2.INTER_NEAREST
    )

    return sampled_image


def quantize_image(image, levels):
    quantized_image = np.floor(
        image / (256 // levels)
    ) * (256 // levels)

    return quantized_image.astype(np.uint8)


image_path = r'C:\Users\M\Desktop\arti\lena_gray_256.tif'

sampling_factor = 10
quantization_levels = 17

original_image = cv2.imread(
    image_path,
    cv2.IMREAD_GRAYSCALE
)

if original_image is None:
    print("Error: Unable to load image")
else:
    sampled_image = sample_image(
        original_image,
        sampling_factor
    )

    quantized_image = quantize_image(
        original_image,
        quantization_levels
    )

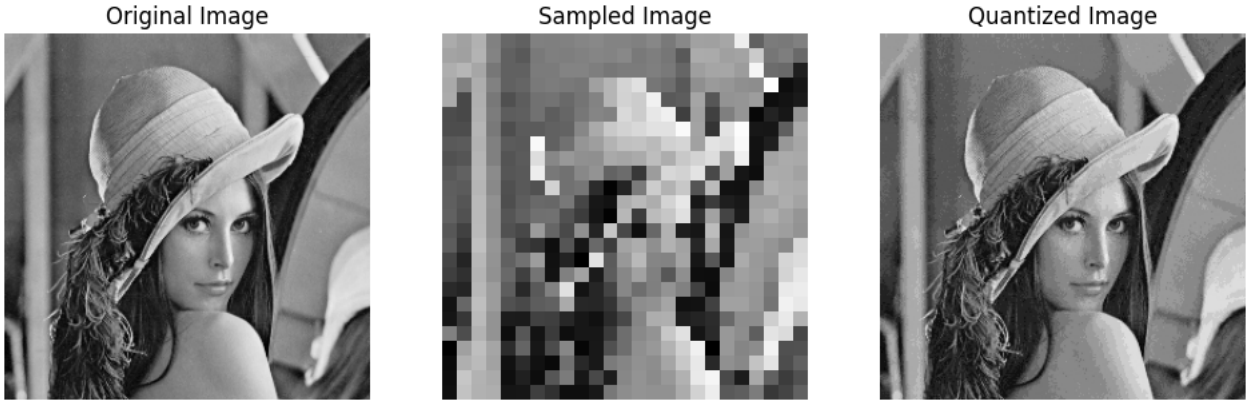
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
```

```
    plt.subplot(1, 3, 2)
    plt.imshow(sampled_image, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized_image, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')

    plt.show()
```



Task #2 — Arithmetic and Set Operations

In [24]:

```
#Step 1 – Read the two images
img1 = Image.open(
    r'C:\Users\M\Desktop\arti\lena_gray_256.tif'
).convert('L')

img2 = Image.open(
    r'C:\Users\M\Desktop\arti\cameraman.tif'
).convert('L')

resize = (400, 400)

img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1)
im2arr = np.asarray(img2)


#1. Subtract two images
subtraction = np.abs(
    im1arr.astype(np.int16) -
    im2arr.astype(np.int16)
)

result = Image.fromarray(subtraction.astype(np.uint8))

plt.imshow(result, cmap='gray')
plt.title('Subtraction')
plt.axis('off')
plt.show()


#2. Add one image with constant 175
addition_175 = np.clip(
    im1arr.astype(np.int16) + 175,
    0,
    255
)

result = Image.fromarray(addition_175.astype(np.uint8))
```

```
plt.imshow(result, cmap='gray')
plt.title('Image + 175')
plt.axis('off')
plt.show()

#3. Set Difference
set_difference = np.abs(
    im1arr.astype(np.int16) -
    im2arr.astype(np.int16)
)

result = Image.fromarray(set_difference.astype(np.uint8))

plt.imshow(result, cmap='gray')
plt.title('Set Difference')
plt.axis('off')
plt.show()

#4. Symmetric Difference
symmetric_difference = np.bitwise_xor(
    im1arr,
    im2arr
)

result = Image.fromarray(symmetric_difference)

plt.imshow(result, cmap='gray')
plt.title('Symmetric Difference')
plt.axis('off')
plt.show()

#5. Intersection
intersection = np.bitwise_and(
    im1arr,
    im2arr
)

result = Image.fromarray(intersection)

plt.imshow(result, cmap='gray')
plt.title('Intersection')
plt.axis('off')
plt.show()
```

Subtraction



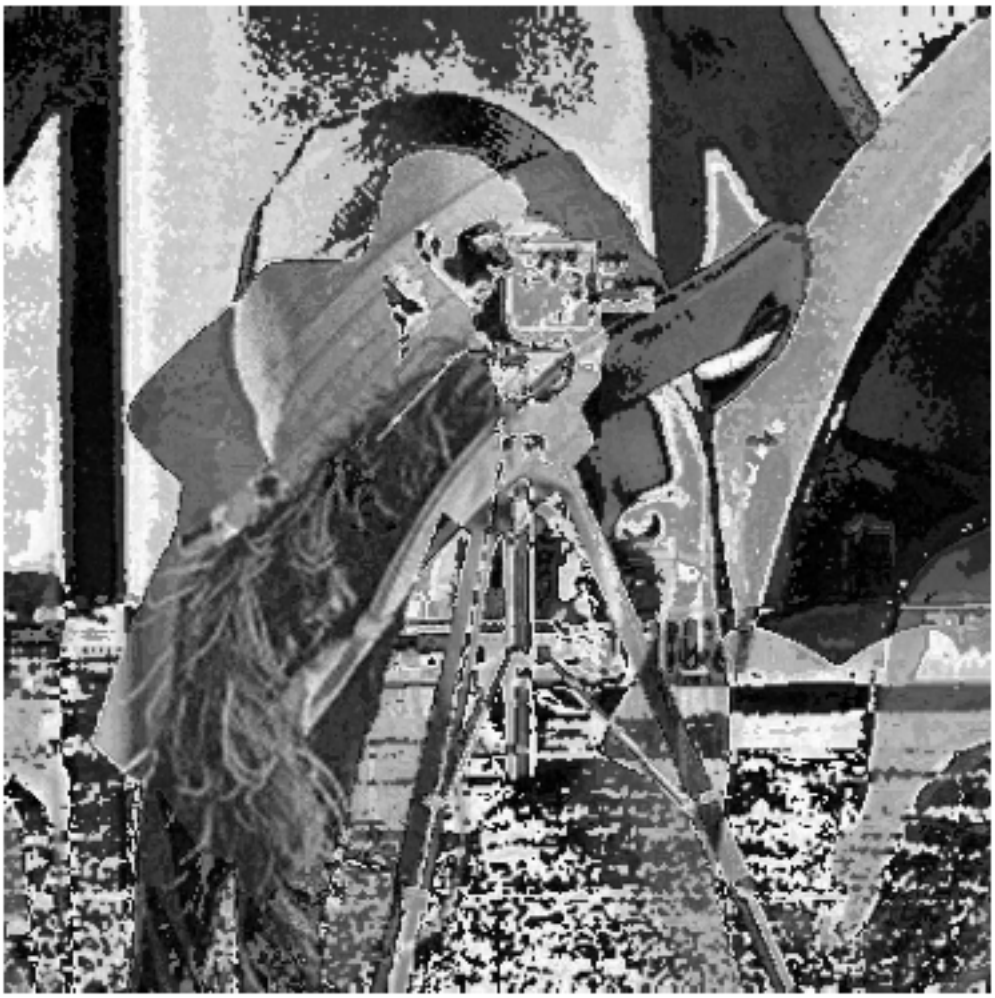
Image + 175



Set Difference



Symmetric Difference



Intersection

